

ANÁLISIS DE SEGURIDAD MEDMANAGER

Sistema de Hashing de Contraseñas

Bcrypt — Implementación Actual y Hoja de Ruta

Autor: Victor Gonzalez

Estado: Seguro — Alineado con mejores prácticas de la industria

Algoritmo: Bcrypt | Salt Rounds: 10 | Estándar: OWASP

1. ¿Qué es Bcrypt y cómo protege las contraseñas?

Cuando un usuario crea su cuenta, su contraseña no se guarda en la base de datos tal como la escribió. En cambio, pasa por **Bcrypt**, un algoritmo de hashing diseñado específicamente para proteger contraseñas. Lo que hace especial a Bcrypt frente a alternativas más simples (MD5, SHA-256) es que está **diseñado intencionalmente para ser lento**: si alguien roba la base de datos e intenta descifrar las contraseñas probando millones de combinaciones, cada intento le toma tiempo computacional real. Lo que para el servidor son ~100ms por login legítimo, para un atacante se convierte en décadas de cómputo por contraseña.

1.1 El flujo completo cuando alguien inicia sesión

1. **El usuario ingresa su contraseña en el formulario de login.** Esta viaja por HTTPS al servidor.
2. **El servidor recupera el hash almacenado en la base de datos** para ese usuario.
3. **Bcrypt aplica el mismo proceso de hashing a la contraseña ingresada** usando el salt guardado junto al hash.
4. **Compara los dos hashes.** Si coinciden, acceso concedido. En ningún momento se "descifra" la contraseña original.

2. ¿Qué pasaría si Bcrypt se usara sin salt?

El salt es la pieza que convierte un hashing funcional en un hashing **realmente seguro**. Sin él, aunque el algoritmo siga siendo Bcrypt, la protección se debilita de manera crítica. A continuación se explica exactamente qué implicaría operar sin salt.

2.1 El problema de los hashes repetidos

Sin salt, la misma contraseña siempre produce el mismo hash. Si en la base de datos hay 500 usuarios y 120 de ellos usan la contraseña "**123456**", los 120 tendrían exactamente el mismo valor en la columna de contraseña. Un atacante que vea ese hash repetido sabe de inmediato que es una contraseña común, y solo necesita crackear ese hash **una sola vez** para comprometer las 120 cuentas simultáneamente.

Con salt, aunque esas 120 personas usen la misma contraseña, cada una tiene un salt único generado al momento de registrarse. El resultado son 120 hashes completamente distintos. El atacante tiene que atacar cada cuenta de forma independiente.

2.2 Rainbow Tables: el arma que el salt destruye

Una **Rainbow Table** es una base de datos precalculada que mapea contraseñas comunes a sus hashes. Existen tablas públicas con miles de millones de entradas. Si tu sistema no usa salt, un atacante puede comparar cada hash de tu base de datos contra esta tabla en cuestión de **segundos**, sin necesidad de computar nada.

Ejemplo concreto de ataque con Rainbow Table

Sin salt:

hash('contraseña123') = \$2b\$10\$XXXXXXXXXXXXXXXXXXXXXXX (siempre el mismo resultado)

El atacante busca ese hash en su tabla → encuentra 'contraseña123' en milisegundos.

Con salt (implementación actual):

hash('contraseña123' + salt_usuario_A) = \$2b\$10\$aB3f... (hash único de A)

hash('contraseña123' + salt_usuario_B) = \$2b\$10\$zK9m... (hash único de B)

La Rainbow Table no sirve: nunca calculó hashes con ese salt específico.

2.3 Un solo ataque exitoso, daño masivo

Sin salt, el daño de un ataque se **multiplica horizontalmente**. Crackear un hash no afecta a una persona, afecta a todas las que usaron esa contraseña. En una base de datos real, las 50 contraseñas más comunes pueden representar el 30–40% de todos los usuarios. El atacante consigue ese porcentaje con 50 operaciones de cracking.

Con salt, crackear la contraseña del usuario A no entrega ninguna información sobre el usuario B, aunque ambos usen la misma. El atacante debe empezar desde cero con cada cuenta. Esto cambia radicalmente el costo del ataque: de **50 operaciones para el 40% de usuarios** a **N operaciones para N usuarios**, donde N es el total de cuentas en el sistema.

2.4 Comparación directa

Sin Salt (hashing simple)	Con Salt (Bcrypt actual)
Dos usuarios con la misma contraseña tienen el mismo hash	Misma contraseña → hashes totalmente distintos
Vulnerable a Rainbow Tables (hashes precalculados)	Rainbow Tables son completamente inútiles
Un hash crackeado expone a todos los usuarios con esa contraseña	Cada hash debe atacarse de forma independiente
El atacante puede precomputar millones de hashes antes del ataque	No hay precomputación posible; todo debe calcularse al momento
Una base de datos robada se puede analizar en masa	Cada cuenta requiere un ataque individual desde cero

Conclusión de esta sección

El salt no es un extra opcional: es lo que hace que el hashing sea resistente a ataques a escala.

Sin él, una sola vulnerabilidad compromete a múltiples usuarios. Con él, cada cuenta es un problema independiente para el atacante. Bcrypt genera el salt automáticamente — la implementación actual lo usa correctamente en todos los puntos del código.

3. La configuración actual: Salt Rounds = 10

En el código ([authRoutes.js](#) y [encriptar.js](#)) Bcrypt está configurado con **saltRounds = 10**. Ese número es el factor de costo: indica cuántas veces itera el algoritmo internamente — concretamente $2^{10} = 1,024$ iteraciones por operación de hashing.

¿Qué significa en la práctica?

- Tiempo por operación de login en el servidor: ~50–100 milisegundos
- Intentos que puede hacer un atacante por segundo (hardware moderno): ~20
- Intentos posibles con SHA-256 sin sal en el mismo hardware: ~20,000,000

- Recomendación OWASP (2024): factor 10–12 para aplicaciones estándar

4. Pros y contras del sistema actual

✓ Ventajas	✗ Limitaciones
✓ Estándar de la industria reconocido globalmente	✗ Dos librerías instaladas (bcrypt y bcryptjs) sin estandarizar
✓ Resistente a fuerza bruta y ataques de diccionario	✗ Factor 10 podría quedar corto frente a hardware futuro más potente
✓ Protección total contra Rainbow Tables vía salt automático	✗ bcryptjs (JS puro) es más lenta que la nativa bcrypt en validaciones
✓ Factor de costo 10 equilibra seguridad y rendimiento	✗ No hay pepper implementado para defensa en profundidad adicional
✓ El factor de costo es ajustable sin invalidar contraseñas existentes	✗ Sin mecanismo de re-hashing automático si se sube el factor de costo
✓ Salt único por usuario generado automáticamente por Bcrypt	✗ Depende de librería nativa (bcrypt) que puede dar problemas en algunos entornos

5. Mejoras recomendadas a futuro

El sistema no requiere cambios urgentes. Las siguientes mejoras son iterativas y pueden aplicarse de forma independiente:

5.1 Estandarizar a una sola librería

Actualmente coexisten **bcrypt** (nativa en C/C++) y **bcryptjs** (JavaScript puro). Ambas producen hashes compatibles, pero tener las dos es deuda técnica innecesaria. Conservar solo **bcrypt** (la nativa) reduce dependencias y mejora el rendimiento en validaciones. Esfuerzo estimado: 1–2 horas.

5.2 Subir salt rounds a 12

La recomendación actual de OWASP apunta a tiempos de ~250ms por operación, equivalente a un factor **12** en hardware moderno. El cambio es una línea de código y no invalida contraseñas existentes: se re-hashean automáticamente en el próximo login de cada usuario. Esfuerzo estimado: 30 minutos.

5.3 Implementar pepper

Un **pepper** es un secreto adicional almacenado en variables de entorno del servidor (no en la base de datos) que se mezcla con la contraseña antes de hashear. Agrega defensa en profundidad: si alguien obtiene el dump de la base de datos pero no tiene acceso al servidor, los hashes son inútiles sin el pepper. Esfuerzo estimado: 2–3 horas.

Resumen Ejecutivo

ESTADO ACTUAL: La implementación de Bcrypt con salt automático y factor 10 es correcta y segura. No hay acción urgente requerida.

EL SALT ES CRÍTICO: Sin él, un solo hash crackeado puede comprometer múltiples cuentas simultáneamente y las Rainbow Tables se vuelven un vector de ataque viable.

MEJORAS RECOMENDADAS EN ORDEN:

1. Unificar a bcrypt nativo (eliminar bcryptjs) [~2 horas]
2. Subir salt rounds de 10 a 12 [~30 minutos]
3. Agregar pepper en variables de entorno [~3 horas]